

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ
по лабораторной работе №2
по дисциплине “Логические основы интеллектуальных систем”
Вариант 2

Выполнил:
Аникеенко А. Д. , гр. 421701
Проверил:
Ивашенко В.П.

Тема: Логическое программирование поиска решения задачи.

Цель: Преобрести навыки логического программирования поиска решения задачи.

Постановка задачи (вариант 2): Задача о переправе (человек, волк, коза, капуста). Имеется два берега реки и лодка, вмещающая человека и не более одного дополнительного объекта (животного или предмета). Задаются произвольные начальное и целевое состояния расположения объектов на берегах. Требуется найти кратчайшую последовательность перемещений для достижения целевого состояния, при условии соблюдения правил безопасности: ни на одном из берегов без присутствия человека не должны оставаться волк с козой, а также коза с капустой.

Дополнительные теоретические сведения:

Задача сводится к поиску на ориентированном графе пространств состояний, где текущее состояние формализуется составным термом, хранящим списки объектов на берегах и позицию лодки. Стандартный механизм логического вывода Prolog (поиск в глубину) для данной задачи неэффективен, так как на графах с циклами он может приводить к заикливанию и не гарантирует нахождения кратчайшего пути. В связи с этим в реализации применен алгоритм поиска в ширину, который гарантированно находит кратчайший маршрут до целевого состояния, а предотвращение заикливаний обеспечивается механизмом отсечения ранее посещенных вершин графа.

Грамматика языка PROLOG:

<Prolog -предложение> ::= <правило> | <факт> | <запрос>

<правило> ::= <заголовок> ‘:-’<тело>

<факт> ::= <заголовок> ‘.’

<запрос> ::= <тело> ‘.’

<тело> ::= <цель> /’,’<цель>/’.’

<заголовок> ::= <предикат>

<цель> ::= <предикат> | <выражение>

<предикат> ::= <имя> / ‘(’<терм> /’,’<терм>/ ‘)’/

<терм> ::= <атом> | <предикат> | <список>

<атом> ::= <переменная> | <число> | <строка> | <имя>

<простой список>::= '[' <терм >/','<терм>/']'|'['']'

<выражение>::= <терм> /<оператор><терм>/

<список с заголовком >::= '[' <терм >/','<терм>/']' < терм>']'

<оператор>::= 'is' | '=' | '==' | '\=' | '>=' | '<=' | '=\\=' |

Использованные встроенные предикаты Prolog:

- **member**(Elem, List) — проверяет, является ли Elem элементом списка List.
- **append**(List1, List2, ResultList) — объединяет списки List1 и List2, возвращая результат в ResultList.
- **findall**(Template, Goal, Bag) — собирает в список Bag все элементы Template, для которых успешно выполняется условие Goal.
- **msort**(List, Sorted) — сортирует элементы списка List с сохранением дубликатов.
- **reverse**(List, Reversed) — обращает порядок элементов списка.
- **length**(List, Length) — вычисляет количество элементов в списке.

Использованные факты:

- **remove**(_, [], []) — используется как базовый случай рекурсивного предиката удаления элемента из списка.
- **remove_list**([], L, L) — используется как базовый случай рекурсивного предиката удаления нескольких элементов из списка.
- **all_objects**([man, wolf, goat, cabbage]) — факт, задающий исчерпывающий список всех допустимых объектов предметной области.

Используемые правила:

- **safe**(Bank) — проверяет безопасность заданного берега.
- **move**(CurrentState, NextState, Carry) — определяет логику перехода системы из текущего состояния в следующее.
- **remove**(X, List, Result) — рекурсивное правило удаления заданного элемента X из списка List.
- **remove_list**(Elements, List, Result) — рекурсивная обертка над remove/3 для последовательного удаления сразу нескольких элементов (человека и его груза) из списка берега.
- **normalized**(RawState, NormalizedState) — приводит состояние системы к каноническому (нормализованному) виду.
- **bfs**(Queue, Goal, Solution) — реализует алгоритм поиска в ширину.

- $\text{solve}(\text{InitState}, \text{GoalState}, \text{Solution})$ — предикат-обертка для запуска алгоритма.

Логические связки, используемые в коде программы:

- $,$ - обозначение конъюнкции.
- $;$ - обозначение дизъюнкции.
- $:-$ - обозначение импликации.
- $\backslash +$ - обозначение отрицания.
- $\rightarrow$ — обозначение условного перехода.

Реализованные предикаты:

1. $\forall \text{Bank} ((\neg \text{member}(\text{man}, \text{Bank}) \rightarrow (\neg(\text{member}(\text{wolf}, \text{Bank}) \wedge \text{member}(\text{goat}, \text{Bank})) \wedge \neg(\text{member}(\text{goat}, \text{Bank}) \wedge \text{member}(\text{cabbage}, \text{Bank})))) \rightarrow \text{safe}(\text{Bank}))$
2. $\forall \text{Left} \forall \text{Right} \forall \text{Carry} \forall \text{NewLeft} \forall \text{NewRight} ((\text{member}(\text{man}, \text{Left}) \wedge (\text{Carry} = \text{none} \vee (\text{member}(\text{Carry}, \text{Left}) \wedge \text{Carry} \neq \text{man}))) \wedge \text{remove_list}([\text{man}, \text{Carry}], \text{Left}, \text{NewLeft}) \wedge \text{NewRight} = [\text{man}, \text{Carry} \mid \text{Right}] \wedge \text{safe}(\text{NewLeft}) \wedge \text{safe}(\text{NewRight})) \rightarrow \text{move}(\text{state}(\text{Left}, \text{Right}, \text{left}), \text{state}(\text{NewLeft}, \text{NewRight}, \text{right}), \text{Carry}))$
3. $\forall \text{Left} \forall \text{Right} \forall \text{Carry} \forall \text{NewLeft} \forall \text{NewRight} ((\text{member}(\text{man}, \text{Right}) \wedge (\text{Carry} = \text{none} \vee (\text{member}(\text{Carry}, \text{Right}) \wedge \text{Carry} \neq \text{man}))) \wedge \text{remove_list}([\text{man}, \text{Carry}], \text{Right}, \text{NewRight}) \wedge \text{NewLeft} = [\text{man}, \text{Carry} \mid \text{Left}] \wedge \text{safe}(\text{NewLeft}) \wedge \text{safe}(\text{NewRight})) \rightarrow \text{move}(\text{state}(\text{Left}, \text{Right}, \text{right}), \text{state}(\text{NewLeft}, \text{NewRight}, \text{left}), \text{Carry}))$
4. $\forall \text{L} \forall \text{R} \forall \text{B} \forall \text{SL} \forall \text{SR} ((\text{msort}(\text{L}, \text{SL}) \wedge \text{msort}(\text{R}, \text{SR})) \rightarrow \text{normalized}(\text{state}(\text{L}, \text{R}, \text{B}), \text{state}(\text{SL}, \text{SR}, \text{B})))$
5. $\forall \text{Init} \forall \text{Goal} \forall \text{Sol} \forall \text{NInit} \forall \text{NGoal} \forall \text{RevPath} ((\text{normalized}(\text{Init}, \text{NInit}) \wedge \text{normalized}(\text{Goal}, \text{NGoal}) \wedge \text{bfs}([\text{NInit}], \text{NGoal}, \text{RevPath}) \wedge \text{reverse}(\text{RevPath}, \text{Sol})) \rightarrow \text{solve}(\text{Init}, \text{Goal}, \text{Sol}))$
6. $\forall \text{GoalNorm} \forall \text{Rest} (\text{bfs}([\text{GoalNorm} \mid \text{Rest}] \mid _), \text{GoalNorm}, [\text{GoalNorm} \mid \text{Rest}])$
7. $\forall \text{Current} \forall \text{Visited} \forall \text{Queue} \forall \text{Goal} \forall \text{Sol} \forall \text{NewPaths} \forall \text{NewQueue} ((\text{findall}([\text{Next}, \text{Current} \mid \text{Visited}], (\text{move}(\text{Current}, \text{NextRaw}, _) \wedge \text{normalized}(\text{NextRaw}, \text{Next}) \wedge \neg \text{member}(\text{Next}, [\text{Current} \mid \text{Visited}])), \text{NewPaths}) \wedge \text{append}(\text{Queue}, \text{NewPaths}, \text{NewQueue}) \wedge \text{bfs}(\text{NewQueue}, \text{Goal}, \text{Sol})) \rightarrow \text{bfs}([\text{Current} \mid \text{Visited}] \mid \text{Queue}, \text{Goal}, \text{Sol}))$
8. $\forall \text{L} \forall \text{R} \forall \text{B} \forall \text{Rest} \forall \text{SL} \forall \text{SR} ((\text{msort}(\text{L}, \text{SL}) \wedge \text{msort}(\text{R}, \text{SR}) \wedge \text{print_solution}(\text{Rest})) \rightarrow \text{print_solution}([\text{state}(\text{L}, \text{R}, \text{B}) \mid \text{Rest}]))$
9. $\forall \text{X} (\text{remove}(\text{X}, [], []))$
10. $\forall \text{X} \forall \text{T} (\text{remove}(\text{X}, [\text{X} \mid \text{T}], \text{T}))$
11. $\forall \text{L} (\text{remove_list}([], \text{L}, \text{L}))$

Код программы:

% Лабораторная работа №2 по дисциплине Логические Основы Интеллектуальных Систем

% Выполнена студентом группы 421701 БГУИР Аникеев Андрей Дмитриевич

% Задание:

% Два берега реки. На одном из них человек, который несёт капусту, ведёт козу и пойманного волка.

% Требуется с помощью лодки, вмещающей вместе с человеком не более одного животного или предмета, переправиться на другой берег.

% Человек не может оставлять козу с капустой и волка с козой.

%

% Используемые источники:

% Zuse, K. (Конрад Цузе) Der Plankalkül [плановое исчисление] / K. Zuse // Gesellschaft für Mathematik und Datenverarbeitung

% [Общество математики и обработки данных]. – 1972.

% SWI-Prolog Documentation: predicate member/2 [Электронный ресурс]. – Режим доступа: <https://www.swi-prolog.org/pldoc/man?predicate=member/2>.

% SWI-Prolog Documentation: Built-in Predicates (append, findall, msort, reverse) [Электронный ресурс].

% – Режим доступа: <https://www.swi-prolog.org/pldoc/man?section=builtin>.

% --- Безопасность берега ---

safe(Bank) :-

(member(man, Bank) -> true

;

\+ (member(wolf, Bank), member(goat, Bank)),

\+ (member(goat, Bank), member(cabbage, Bank))

).

% --- Переходы ---

```

move(state(Left, Right, left), state(NewLeft, NewRight, right), Carry) :-
    member(man, Left),
    ( Carry = none ; (member(Carry, Left), Carry \= man) ),
    ( Carry = none -> remove(man, Left, NewLeft)
    ; remove_list([man, Carry], Left, NewLeft) ),
    ( Carry = none -> NewRight = [man | Right]
    ; NewRight = [man, Carry | Right] ),
    safe(NewLeft),
    safe(NewRight).

```

```

move(state(Left, Right, right), state(NewLeft, NewRight, left), Carry) :-
    member(man, Right),
    ( Carry = none ; (member(Carry, Right), Carry \= man) ),
    ( Carry = none -> remove(man, Right, NewRight)
    ; remove_list([man, Carry], Right, NewRight) ),
    ( Carry = none -> NewLeft = [man | Left]
    ; NewLeft = [man, Carry | Left] ),
    safe(NewLeft),
    safe(NewRight).

```

% --- Вспомогательные ---

```

remove(_, [], []).
remove(X, [X|T], T) :- !.
remove(X, [H|T], [H|R]) :- remove(X, T, R).

```

```

remove_list([], L, L).
remove_list([H|T], L, R) :-
    remove(H, L, L1),

```

```
remove_list(T, L1, R).
```

```
normalized(state(L, R, B), state(SL, SR, B)) :-  
    msort(L, SL), msort(R, SR).
```

```
% --- BFS ---
```

```
solve(InitState, GoalState, Solution) :-  
    normalized(InitState, NInit),  
    normalized(GoalState, NGoal),  
    bfs([[NInit]], NGoal, RevPath),  
    reverse(RevPath, Solution).
```

```
bfs([[GoalNorm | Rest] | _], GoalNorm, [GoalNorm | Rest]) :- !.
```

```
bfs([[Current | Visited] | Queue], Goal, Solution) :-
```

```
    findall(  
        [Next, Current | Visited],  
        (  
            move(Current, NextRaw, _),  
            normalized(NextRaw, Next),  
            \+ member(Next, [Current | Visited])  
        ),  
        NewPaths  
    ),  
    append(Queue, NewPaths, NewQueue),  
    bfs(NewQueue, Goal, Solution).
```

```
% --- Печать ---
```

```
print_solution([]) :- nl.
```

```

print_solution([state(L, R, B) | Rest]) :-
    msort(L, SL), msort(R, SR),
    write(' Левый берег : '), write(SL),
    write(' | Лодка: '), write(B),
    write(' | Правый берег: '), write(SR), nl,
    print_solution(Rest).

% --- Ввод строками ---

% Читаем строку и парсим её как Prolog-терм
read_term_from_line(Term) :-
    read_line_to_string(user_input, Line),
    % Добавляем точку если пользователь не поставил
    ( string_concat(_, ".", Line) ->
        TermStr = Line
    ;
        string_concat(Line, ".", TermStr)
    ),
    term_to_atom(Term, TermStr).

% Читаем yes/no
ask_yes_no(Question, Answer) :-
    repeat,
        format("~w (yes/no): ", [Question]),
        read_line_to_string(user_input, Line),
        string_lower(Line, Low),
        ( Low = "yes" -> Answer = yes, !
        ; Low = "no" -> Answer = no, !
        ;
            write(' Введите yes или no'), nl,

```

fail

).

% Читаем список объектов

% Допустимые: man, wolf, goat, cabbage

all_objects([man, wolf, goat, cabbage]).

read_bank(Prompt, List) :-

repeat,

format("~w~n", [Prompt]),

write(' Допустимые объекты: man wolf goat cabbage'), nl,

write(' Введите через пробел (или оставьте пустым для пустого берега): '),

read_line_to_string(user_input, Line),

split_string(Line, " ", " ", Parts),

include([P]>>(P \= ""), Parts, NonEmpty),

maplist(atom_string, Atoms, NonEmpty),

all_objects(Allowed),

(maplist([A]>>(member(A, Allowed)), Atoms) ->

List = Atoms, !

;

write(' Ошибка: недопустимые объекты. Попробуйте снова. '), nl,

fail

).

% Читаем сторону лодки

read_side(Prompt, Side) :-

repeat,

format("~w (left/right): ", [Prompt]),

read_line_to_string(user_input, Line),

string_lower(Line, Low),

```
( Low = "left" -> Side = left, !
; Low = "right" -> Side = right, !
;
write(' Ошибка: введите left или right'), nl,
fail
).
```

% Валидация: man ровно на одном берегу

validate_man(Left, Right) :-

```
( member(man, Left), \+ member(man, Right) -> true
; member(man, Right), \+ member(man, Left) -> true
;
write(' Ошибка: man должен быть ровно на одном берегу.'), nl,
fail
).
```

% Валидация: нет дублей между берегами

no_duplicates(Left, Right) :-

```
( \+ (member(X, Left), member(X, Right)) -> true
;
write(' Ошибка: один объект не может быть на обоих берегах.'), nl,
fail
).
```

% Читаем полное состояние

read_state(Label, state(Left, Right, Boat)) :-

```
repeat,
format("~n=== ~w ===~n", [Label]),
read_bank('Левый берег:', Left),
read_bank('Правый берег:', Right),
```

```

read_side('Сторона лодки', Boat),
( validate_man(Left, Right), no_duplicates(Left, Right) ->
    !
;
    write(' Некорректное состояние, повторите ввод.'), nl,
    fail
).

```

% --- Главный предикат ---

main :-

```

write('Ограничения:'), nl,
write(' - Лодка вмещает человека + не более 1 объекта'), nl,
write(' - Нельзя оставлять волка с козой без человека'), nl,
write(' - Нельзя оставлять козу с капустой без человека'), nl,
nl,
ask_yes_no('Использовать стандартное начальное состояние?', UseDefault),
nl,
( UseDefault = yes ->
    Init = state([man, wolf, goat, cabbage], [], left),
    Goal = state([], [man, wolf, goat, cabbage], right),
    write('Начальное: все на левом берегу.'), nl,
    write('Целевое: все на правом берегу.'), nl
;
    read_state('НАЧАЛЬНОЕ СОСТОЯНИЕ', Init),
    read_state('ЦЕЛЕВОЕ СОСТОЯНИЕ', Goal)
),
nl,
write('Поиск решения...'), nl, nl,
( solve(Init, Goal, Solution) ->

```

```
length(Solution, Len),
```

```
Steps is Len - 1,
```

```
format("Решение найдено за ~w шагов:~n~n", [Steps]),
```

```
print_solution(Solution)
```

```
;
```

```
write('Решение не найдено для заданных состояний. '), nl
```

```
).
```

```
:- initialization(main, main).
```

Проверка работоспособности:

В ходе тестирования был запущен основной алгоритм поиска решения задачи о переправе. Программа успешно обработала запрос, используя стандартные начальные условия (все объекты на левом берегу), и нашла кратчайшую последовательность переходов, состоящую из 7 шагов. Результат выполнения программы, демонстрирующий пошаговое изменение состояния берегов и положение лодки:

```
Задача о переправе: волк, коза, капуста
Вариант 2

Ограничения:
- Лодка вмещает человека + не более 1 объекта
- Нельзя оставлять волка с козой без человека
- Нельзя оставлять козу с капустой без человека

Использовать стандартное начальное состояние? (yes/no): yes

Начальное: все на левом берегу.
Целевое:   все на правом берегу.

Поиск решения...

Решение найдено за 7 шагов:

Левый берег : [cabbage,goat,man,wolf] | Лодка: left | Правый берег: []
Левый берег : [cabbage,wolf] | Лодка: right | Правый берег: [goat,man]
Левый берег : [cabbage,man,wolf] | Лодка: left | Правый берег: [goat]
Левый берег : [wolf] | Лодка: right | Правый берег: [cabbage,goat,man]
Левый берег : [goat,man,wolf] | Лодка: left | Правый берег: [cabbage]
Левый берег : [goat] | Лодка: right | Правый берег: [cabbage,man,wolf]
Левый берег : [goat,man] | Лодка: left | Правый берег: [cabbage,wolf]
Левый берег : [] | Лодка: right | Правый берег: [cabbage,goat,man,wolf]
```

Рис. 1. Результат работы алгоритма.

Для демонстрации корректной обработки исключительных ситуаций было проведено тестирование модуля ввода данных. В случае попытки ввода недопустимого объекта (например, «cat» вместо разрешенных «man», «wolf», «goat», «cabbage»), программа корректно идентифицирует ошибку, уведомляет пользователя и запрашивает повторный ввод данных:

```
Использовать стандартное начальное состояние? (yes/no): no

=== НАЧАЛЬНОЕ СОСТОЯНИЕ ===
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): cat
Ошибка: недопустимые объекты. Попробуйте снова.
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега):
```

Рис. 2. Демонстрация валидации.

Протокол выполнения программы, демонстрирующий обработку состояний, для которых путь в графе решений отсутствует. Алгоритм штатно завершает поиск и возвращает отрицательный результат при невозможности достижения целевой вершины в соответствии с заданными ограничениями:

```
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): cat
Ошибка: недопустимые объекты. Попробуйте снова.
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): wolf man
Правый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): goat cabbage
Сторона лодки (left/right): left

=== ЦЕЛЕВОЕ СОСТОЯНИЕ ===
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега):
Правый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): man wolf goat cabbage
Сторона лодки (left/right): left

Поиск решения...

Решение не найдено для заданных состояний.
```

Рис. 3. Результат работы алгоритма при отсутствии допустимого решения.

Нахождение кратчайшего пути для произвольно заданных состояний. Программа вычисляет последовательность переходов, соответствующую ограничениям безопасности:

```
=== НАЧАЛЬНОЕ СОСТОЯНИЕ ===
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): man wolf cabbage
Правый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): goat
Сторона лодки (left/right): left

=== ЦЕЛЕВОЕ СОСТОЯНИЕ ===
Левый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): goat
Правый берег:
Допустимые объекты: man wolf goat cabbage
Введите через пробел (или оставьте пустым для пустого берега): man wolf cabbage
Сторона лодки (left/right): right

Поиск решения...

Решение найдено за 3 шагов:

Левый берег : [cabbage,man,wolf] | Лодка: left | Правый берег: [goat]
Левый берег : [wolf] | Лодка: right | Правый берег: [cabbage,goat,man]
Левый берег : [goat,man,wolf] | Лодка: left | Правый берег: [cabbage]
Левый берег : [goat] | Лодка: right | Правый берег: [cabbage,man,wolf]
```

Рис. 4. Результат для пользовательского сценария.

Протокол выполнения программы, демонстрирующий невозможность нахождения решения. Заданное целевое состояние признано алгоритмом недопустимым, что привело к завершению поиска без вывода пути:

```
=== НАЧАЛЬНОЕ СОСТОЯНИЕ ===
Левый берег:
  Допустимые объекты: man wolf goat cabbage
  Введите через пробел (или оставьте пустым для пустого берега): man wolf cabbage
Правый берег:
  Допустимые объекты: man wolf goat cabbage
  Введите через пробел (или оставьте пустым для пустого берега): goat
Сторона лодки (left/right): left

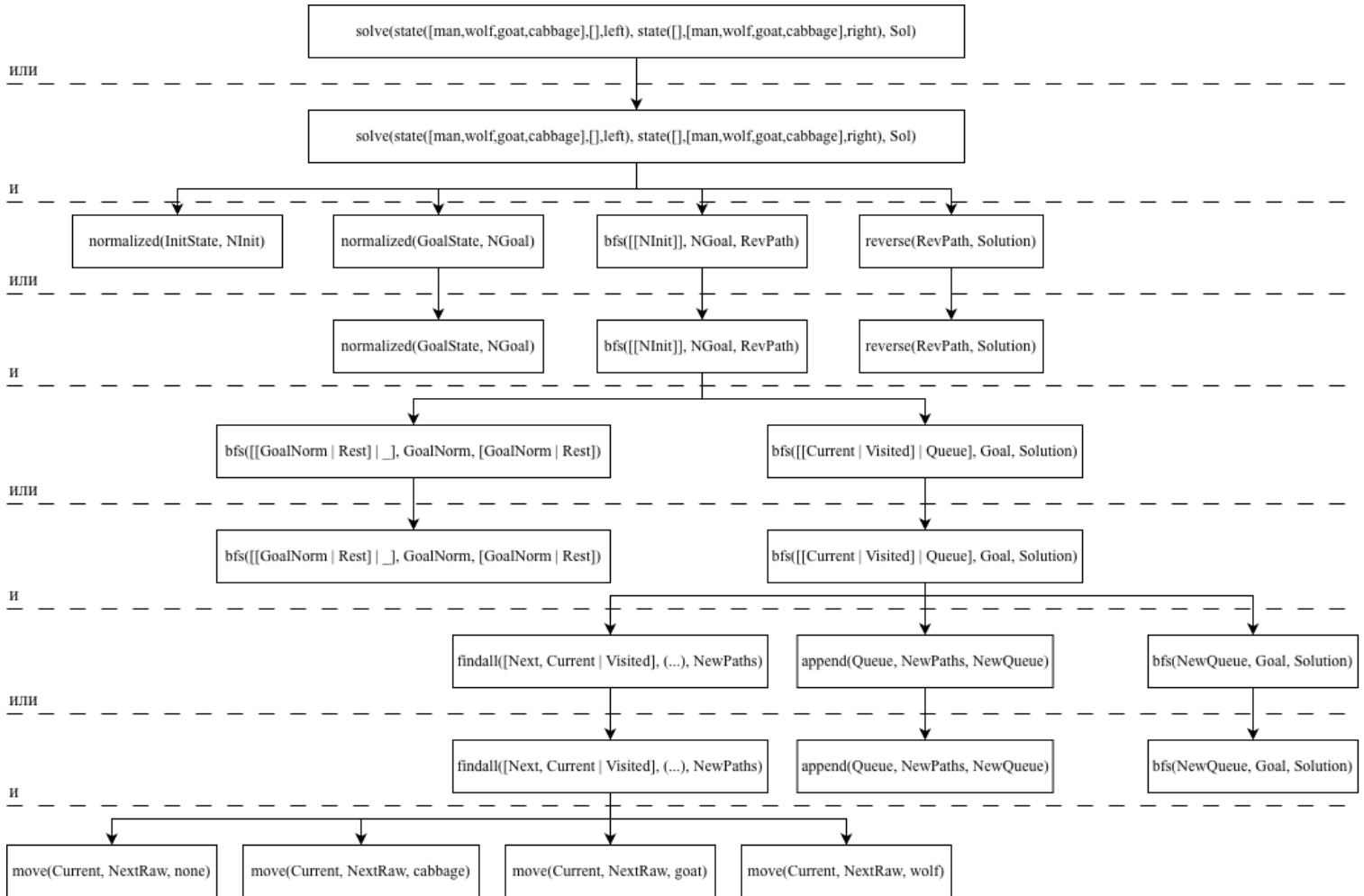
=== ЦЕЛЕВОЕ СОСТОЯНИЕ ===
Левый берег:
  Допустимые объекты: man wolf goat cabbage
  Введите через пробел (или оставьте пустым для пустого берега): man wolf
Правый берег:
  Допустимые объекты: man wolf goat cabbage
  Введите через пробел (или оставьте пустым для пустого берега): goat cabbage
Сторона лодки (left/right): left

Поиск решения...

Решение не найдено для заданных состояний.
```

Рис. 5. Обработка недостижимого целевого состояния.

Дерево вывода предиката поиска решений задачи о переправе:



Вывод:

В рамках данной лабораторной работ были приобретены навыки логического программирования поиска решения задачи на языке Prolog. Также были расширены знания о декларативном программировании.

Использованные источники:

Zuse, K. (Конрад Цузе) Der Plankalkül [плановое исчисление] / K. Zuse // Gesellschaft für Mathematik und Datenverarbeitung [Общество математики и обработки данных]. – 1972.

SWI-Prolog Documentation: predicate member/2 [Электронный ресурс]. – Режим доступа: <https://www.swi-prolog.org/pldoc/man?predicate=member/2>.

SWI-Prolog Documentation: Built-in Predicates (append, findall, msort, reverse) [Электронный ресурс]. – Режим доступа: <https://www.swi-prolog.org/pldoc/man?section=builtin>.